ayene-matrix - Full Technical Report

Live Face Mirror with OSC, Wekinator and Streaming

Mohammad Keshmiri

Academic year 2025/2026
Politecnico di Torino

This is the extended technical report. The short 1–2 page summary for submission is `REPORT.pdf`. *The editable markdown source is* `REPORT.md`.

1 Goal

The goal of this project was to build a live interactive *mirror*. The system reads my face from the webcam, classifies my expression into one of four moods, and draws my face as a dot-matrix portrait that changes color, shape and motion with the mood. The final image is streamed locally to a web page, so the work can be shown as a small performance.

The word *ayene* means mirror in Persian. The project is not meant to be a realistic face filter, but a digital reflection: my expression becomes the controller of the image.

I built this as a **solo project**. Following the assignment rules, the two software entities run as independent processes on the same machine and communicate over `localhost`. The system has four parts: sensing, machine learning, rendering, and streaming.

2 My approach

Before writing the final version I tried several designs. I did not jump directly to the working pipeline, and documenting these steps helped me understand what to defend in the oral exam.

What I planned at the beginning

My approved concept was **Ayene / Specchio**: a cultural mirror driven by facial expression. I first considered a different project based on hand gestures, bilingual text and audio mixing, but during prototyping it felt less stable and harder to control in a live setting. The face mirror had a clearer structure: one input (my face), one classifier (four moods), one visual language.

My first technical plan was:

1. Python reads the webcam and extracts face features.
2. Wekinator classifies the features into four moods.
3. Processing draws an abstract portrait that reacts to the mood.
4. OBS captures Processing and publishes the result through a self-hosted MediaMTX server to a browser page.

This is the same input–Wekinator–output structure I already used in Lab 2, Lab 3 Exercise 1 and Lab 3 Exercise 2, but applied to facial expression instead of sound, mouse or hand landmarks.

Version 1 - geometric landmark features

In the first implementation I used MediaPipe face landmarks and built my own features by measuring facial distances such as mouth width, mouth height, eyebrow distance, eye openness, and similar values. I normalized horizontal distances by eye width and vertical distances by nose length, and I added a short calibration step on my neutral face.

This version was **more creative and more transparent**, because every input was a real geometric measurement I could point to on the screen. But it was not reliable enough for a live demo. When I moved my head up, the classifier often detected Surprised; when I moved my head down, it detected Happy, even if my expression did not change. The distances were changing with head pose, not only with expression. Normalization and calibration helped, but the system still needed per-person calibration.

Version 2 - MediaPipe blendshapes (final sensing)

For the final version I switched to MediaPipe **blendshapes**. These are expression scores the model already computes (for example `mouthSmileLeft`, `jawOpen`, `browDownLeft`), each between 0 and 1. I had already tested them successfully in my earlier `ayene-anti` prototype, so I reused the same idea here.

I still send **8 numbers** to Wekinator, but now they are built from blendshape scores instead of raw landmark distances. This removed calibration completely and made the four moods much more stable during head movement.

Version 3 - macOS camera conflict (solved with OSC portrait grid)

At first I wanted both Python and Processing to read the webcam, like in a normal two-sketch lab setup. On my Mac this failed: when `a\_sense.py` was running, Processing received a black camera feed because only one program can open the built-in FaceTime HD camera at a time. This blocked the demo until I changed the design.

Solution. Only `a\_sense.py` opens the camera now. It sends two OSC streams:

- `/wek/inputs` to Wekinator (8 expression features);
- `/ayene/grid` to Processing (a head-shaped brightness grid as a byte blob).

Processing no longer captures video itself. It only listens on port `12000` and draws the portrait from the grid. After this fix, both entities run together reliably in the live demo.

Version 4 - visual design iterations

The rendering also changed during development:

- **Neutral:** calm blue circles. This stayed from the start.
- **Happy:** gold circles with a small wave motion. This also stayed.
- **Surprised:** first I tried cyan rays bursting from the center. I changed this to **pulsing dots** with a staggered ripple, because it read more clearly as a "gasp" on the dot matrix.

- **Focused:** first I used purple squares plus a white scan line. I removed the scan line and kept only sharp purple squares, because the scan line was distracting during the live demo.

I also added a **head-shape mask** in Python using the MediaPipe face outline (`FACE\_OVAL`). Only pixels inside the real head contour are sent to Processing, so the mirror shows my head as dots and not my shoulders or the room behind me.

3 Version comparison

Part	First attempt	Final version
Features	landmark distances + calibration	8 blendshape scores, no calibration
Face model API	old `mp.solutions` idea, then Tasks API	MediaPipe Face Landmarker Tasks API
Camera use	Python and Processing both wanted the webcam	only Python uses the webcam
Portrait data	Processing `Video` capture	OSC `/ayene/grid` byte blob
Surprised look	cyan rays	cyan pulsing dots
Focused look	squares + scan line	purple squares only
Streaming	OBS + MediaMTX WHIP/WebRTC	fully local, no external ICE server

4 System architecture

The assignment asks for two software entities communicating through OSC. In my design they are:

Entity	File	Job
A	`a\_sense.py` (Python)	read the face, send 8 features and portrait grid
B	`B\_Ayene.pde` (Processing)	draw dot-matrix portrait, react to mood

Wekinator is the machine learning mediator between them. It is not inside either entity, which matches the course structure: sensing, ML, rendering, streaming are separate conceptual parts.

The four moods are:

Class	Mood	Expression I perform
1	Neutral	resting face
2	Happy	smile
3	Surprised	open mouth + raised eyebrows
4	Focused	frown / squint

5 How the data flows

Every frame of the live demo (∼30 Hz) follows the same path on `localhost`. The assignment separates sensing, machine learning, rendering and streaming; below is how they connect in my project.

The four stages

1. **Sensing — webcam to Python**

   Only `a_sense.py` opens the camera. On macOS, a second program (Processing) showed a black feed when both tried to use the webcam, so Python became the sole camera client. MediaPipe Face Landmarker outputs **8 blendshape scores** and a 96×72 head-masked brightness grid every frame.

2. **Machine learning — Python to Wekinator**

   Python sends the 8 scores to Wekinator as OSC on `/wek/inputs`, port `6448` (same convention as the lab exercises). The included Wekinator artifact runs a 4-class KNN classifier (saved $k = 1$) and outputs the mood class on `/wek/outputs`, port `12000`.

3. **Rendering — portrait and mood in Processing**

   `B_Ayene.pde` listens on port `12000` and combines two inputs: the **portrait grid** from Python (`/ayene/grid`) and the **mood class** from Wekinator (`/wek/outputs`). It draws a dot-matrix mirror on black; each mood changes color, shape and motion.

4. **Streaming — Processing window to browser**

   OBS captures the Processing window and publishes with WHIP to MediaMTX. The page `web/index.html` embeds MediaMTX's direct `/live` WebRTC player. Everything runs locally; no external streaming service is used.

OSC messages

All structural data between the two software entities uses OSC:

From → To	Address	Port	Payload
`a_sense.py` → Wekinator	`/wek/inputs`	6448	8 expression scores (floats)
`a_sense.py` → `B_Ayene.pde`	`/ayene/grid`	12000	96×72 grid (byte blob)
Wekinator → `B_Ayene.pde`	`/wek/outputs`	12000	mood class 1–4

Flow in one line: webcam → `a_sense.py` → Wekinator → `B_Ayene.pde` → OBS → MediaMTX → browser (with `/ayene/grid` sent directly from Python to Processing in parallel).

Why `/ayene/grid` exists. I added this address after the macOS camera conflict. Processing draws the mirror from the grid instead of opening the webcam. The grid is sent as bytes (not floats) because 6912 floats in one UDP packet raised `OSError: Message too long`; the byte blob fits the `oscP5` datagram limit (`16384`).

6 The 8 features

I send 8 numbers per frame. Each one is built from blendshape scores, and the left/right pairs are averaged together:

#	Feature	Built from	Associated mood
1	Smile	mouthSmile L/R	Happy
2	JawOpen	jawOpen	Surprised
3	BrowUp	browInnerUp	Surprised
4	BrowDown	browDown L/R	Focused
5	Squint	eyeSquint L/R	Focused
6	EyeWide	eyeWide L/R	Surprised
7	Cheek	cheekSquint L/R	Happy
8	Neutral	1 minus expressive sum	Neutral

Because the values are already between 0 and 1, I do not need calibration. The Python window also shows these 8 values as bars and a guessed mood label, but the real decision is always made by Wekinator.

7 Step by step procedure

This is what I actually run for the demo:

Step 1 - sensing. I run `python3 a\_sense.py`. It opens the webcam, detects the face with MediaPipe Face Landmarker (blendshapes enabled), builds the 8 features, sends `/wek/inputs` to port `6448`, and sends the head-masked portrait grid to `/ayene/grid` on port `12000`. The window shows tracked points, the head outline, and the feature bars.

Step 2 - Wekinator. I open `WekinatorProject/matrix.wekproj`. The settings are: 8 inputs, one classifier output, 4 classes. The included classifier is KNN with saved $k = 1$; it outputs to `/wek/outputs` on port `12000`.

Step 3 - training. For each class I make the matching face and record a few seconds, moving my head slightly so pose does not dominate: class 1 neutral, class 2 smile, class 3 open mouth + raised eyebrows, class 4 frown / squint. Then Train, then Run.

Step 4 - visualization. I run `B\_Ayene.pde`. It listens on port `12000`, reads the mood from Wekinator and the portrait grid from Python, and draws dots on black. Brighter pixels produce bigger dots.

Mood	Color	Shape / motion
Neutral	gray-blue	calm circles
Happy	gold	circles with a small wiggle
Surprised	cyan	pulsing dots (ripple)
Focused	indigo	sharp purple squares

Step 5 - streaming. I run MediaMTX with `mediamtx.yml`, capture the Processing window in OBS, publish with WHIP to `http://127.0.0.1:8889/live/whip`, and open `web/index.html`. Everything stays on localhost, with no external streaming service.

8 Results

The blendshape version classified the four moods reliably and did not need calibration. Neutral, Happy and Surprised were the clearest. Focused was a bit harder because a frown and a squint can look similar to a strong neutral, so I recorded a few extra examples for that class and it improved.

Mood	Result
Neutral	stable, this is the resting state
Happy	very clear, smile and cheek scores are strong
Surprised	very clear; jaw opening and raised eyebrows are easy to separate
Focused	worked after more examples, the weakest of the four

The full chain works together: Python sensing, Wekinator classification, Processing visuals, and browser streaming through MediaMTX.

9 Where I got stuck and how I solved it

Problem 1 - head pose changed the prediction

With geometric landmark features, moving my head up was detected as Surprised and moving it down as Happy.

Why it happened. Raw face distances change when the head rotates, not only when the expression changes.

How I handled it. I first tried normalization and calibration. That helped but still needed per-person tuning. I switched to MediaPipe blendshapes, which are pose-robust by design, and the problem went away.

Problem 2 - calibration did not transfer between people

The geometric version was calibrated to my neutral face, so it did not work well when someone else tried it without recalibrating.

How I handled it. Blendshapes are already normalized expression scores, so the final version works for different people without calibration.

Problem 3 - only one program could use the camera

On my Mac, `a\_sense.py` and `B\_Ayene` could not both read the webcam. Processing showed a black feed whenever Python was already running.

How I solved it. I stopped opening the camera in Processing. Only Python captures video now; it sends the head-shaped brightness grid over OSC as `/ayene/grid`, and Processing draws the mirror from that stream. After this change, both entities run together reliably in the live demo.

Problem 4 - OSC message too long

When I first tried sending the portrait grid as floats, Python raised `OSError: Message too long`.

How I handled it. I packed the 96×72 grid as a byte blob (values 0–255) and increased the `oscP5` datagram size to `16384`.

Problem 5 - OBS could not connect to MediaMTX

At one point OBS failed to publish through WHIP.

How I handled it. I restarted MediaMTX, used `127.0.0.1` for the WHIP endpoint, and verified the stream on MediaMTX's direct `/live` page. Since the demo stays on localhost, it does not require an external STUN server.

10 Discussion

The main lesson for me was the same as in the lab exercises: the most detailed or most "clever" features are not automatically the best in practice. My hand-made geometric features were more creative, but the blendshapes were clearly more reliable for a live interactive system.

There is a trade-off. Geometric features are fully explainable, but they put all the difficulty on me: normalization, calibration, tuning. Blendshapes hide that work inside the model, so I lose some transparency, but I gain stability. For a presentation where things must work in front of people, stability won.

On the rendering side, mapping pixel brightness to dot size keeps the project readable as a mirror. Giving each mood its own color, motion and shape makes the classification visible at a glance. That is important for an HMI project, because the audience must *see* the machine interpretation of the human input.

For streaming, I kept everything self-hosted as required: OBS captures the rendering machine, MediaMTX receives WHIP and serves WHEP to the browser. This is the performative part of the work: the mirror is not only on my laptop screen, it can be watched as a live stream on a web page.

11 Conclusion

This project connected face sensing, machine learning and real-time graphics through OSC. I started with creative geometric landmark features, found they were too sensitive to head pose and calibration, and moved to MediaPipe blendshapes for the final version. I also solved the Mac camera conflict by sending a head-masked portrait grid from Python to Processing.

The final system is appropriate for a solo project: 8 inputs, one trained classifier, four distinct visual moods, and a complete local streaming path. The most important practical result is that the live demo is dependable, while the dot-matrix visualization makes each mood clearly visible.

12 Files delivered

- `a\_sense.py` - Entity A, face sensing and OSC output
- `B\_Ayene/B\_Ayene.pde` - Entity B, dot-matrix rendering
- `WekinatorProject/` - trained Wekinator classifier project
- `face\_landmarker.task` - MediaPipe face model
- `mediamtx.yml` - local WebRTC streaming configuration
- `web/index.html` - browser viewer (WHEP)
- `requirements.txt` - Python dependencies
- `README.md` - setup and run instructions
- `REPORT.pdf` - short summary (1–2 pages)
- `REPORT.md` - full report in markdown
- `REPORT\_full.pdf` - this extended report